



Universidad Autónoma de Tamaulipas

Producto integrador: punto de venta

Codigo java

Facultad de Ingeniería Tampico

Ingeniería en Sistemas Computacionales

Asignatura: Fundamentos De programación

Grupo: N Grado: 1

Nombre del Docente: Álvarez Navarro Eduardo

Alumno: Guevara Martinez Angel Jeremy

Matricula: 2243330342

```
import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStreamReader;

import java.text.SimpleDateFormat;

import java.util.Date;


public class proyecto_integrador {

    static String[][] productos = new String[20][5];


    static String ventas[][];

    static int tamventas = 100;

    static String fecha;


    public static String MostrarMenu(String[] opciones)
    {
        String cadena = "";
        for (String info : opciones)
        {
            cadena = cadena + info + "\n";
        }
        return cadena;
    }


    public static boolean EsNumeroEntero(String dato) {
```

```
for (char c : dato.toCharArray()) {  
    if (!Character.isDigit(c)) {  
        return false;  
    }  
}  
return true;  
}
```

```
public static boolean EsNumeroDouble(String dato) {  
    boolean punto = false;  
    if (dato == null || dato.equals("")) {  
        return false;  
    }  
    for (char c : dato.toCharArray()) {  
        if (Character.isDigit(c)) {  
        } else if (c == '.') {  
            if (!punto) {  
                punto = true;  
            } else {  
                return false;  
            }  
        } else {  
            return false;  
        }  
    }  
}
```

```
}  
    return true;  
}  
  
public static String LeerValidado(String texto_dialogo, int tipo_dato) throws IOException {  
    String cadena = "";  
    do {  
        cadena = Leer(texto_dialogo);  
        if (cadena == null) {  
            System.out.println("dato nulo");  
        } else {  
            if (!EvaluarNumerico(cadena, tipo_dato)) {  
                System.out.println("dato incorrecto");  
                cadena = null;  
            }  
        }  
    } while (cadena == null);  
    return cadena;  
}  
  
public static boolean EvaluarNumerico(String dato, int tipo)  
{  
    boolean valido = false;  
    switch (tipo) {  
        case 1:
```

```
valido = EsNumeroEntero(dato);  
  
break;  
  
case 2:  
valido = EsNumeroDouble(dato);  
break;  
}  
return valido;  
}
```

```
public static String Dialogo(String texto) throws IOException  
{  
String cadena;  
System.out.println(texto + " : ");  
BufferedReader lectura = new BufferedReader(new InputStreamReader(System.in));  
cadena = lectura.readLine();  
return cadena;  
}
```

```
public static String Leer(String texto) throws IOException  
{  
String cadena = "";  
cadena = Dialogo(texto);  
if (cadena != null)  
{  
cadena = cadena.trim();  
}}
```

```
        if (cadena.isEmpty())  
            cadena=null;  
    }  
    else  
        cadena = null;  
  
    return cadena;  
}
```

```
public static String DesplegarMenu(String Titulo1, String[] menu) throws IOException  
{  
    String cadena;  
    cadena=Titulo1 + "\n" + "\n";  
    cadena=cadena+MostrarMenu(menu);  
    cadena = cadena + "\n Que opcion deseas ";  
    return  cadena = Dialogo(cadena);  
}
```

```
public static String RellenarEspacios(String dato, int tamano)  
{  
    return String.format("%1$-" + tamano + "s", dato);  
}
```

```
public static String Fecha() {  
    Date fecha = new Date();  
    SimpleDateFormat formatodia = new SimpleDateFormat("dd-MM-yyyy");
```

```
return formatodia.format(fecha);  
}
```

```
public static String IdTicketSiguiente(String idticket) {  
    String idticketnext = "";  
    int num = Integer.parseInt(idticket) + 1;  
    if (num < 10)  
    { idticketnext = "00" + String.valueOf(num).trim(); }  
    else if ((num > 9) && (num < 100))  
    { idticketnext = "0" + String.valueOf(num).trim();  
    }  
    else {  
        idticketnext = String.valueOf(num).trim();  
    }  
    return idticketnext;  
}
```

```
public static int ObtenerUltimaPosicion(String[][] matriz) {  
    int ultimaPosicion = -1; // Inicializa la última posición como -1 (indicando que no se encontró una  
                               // posición válida)  
    for (int i = 0; i < matriz.length; i++) {  
        // Verifica si la primera columna de la fila actual no es nula ni está vacía  
        if (matriz[i][0] != null && !matriz[i][0].isEmpty()) {
```

```
        ultimaPosicion = i; // Actualiza la última posición al índice actual
    }
}
return ultimaPosicion; // Devuelve la última posición válida encontrada
}
```

```
public static String[][] CargarProductos() {
```

```
    String[][] producto = {
```

```
        {"001", "Frappe Chocolate", "65", "20", "16"},
        {"002", "Frappe Vainilla", "65", "20", "16"},
        {"003", "Capuchino Combo", "119", "15", "16"},
        {"004", "Latte Combo", "119", "15", "16"},
        {"005", "Lechero Combo", "119", "15", "16"},
        {"006", "Galletas 2x", "80", "25", "16"},
        {"007", "Rebanada de Pastel", "59", "18", "16"},
        {"008", "Tes y Tisanas 2x", "99", "20", "16"},
        {"009", "Panini Tradicional", "125", "10", "16"},
        {"010", "Bisquet con Mantequilla", "89", "15", "16"},

        {"011", "Ramen Coreano Picante", "85", "12", "16"},
        {"012", "Ramen Coreano Queso", "85", "12", "16"},
        {"013", "Ramen Coreano Pollo", "85", "12", "16"},
        {"014", "Ramen Coreano Carne", "85", "12", "16"},
        {"015", "Bebida Fria", "35", "20", "16"},
```

```
{"016", "Sandwich Jamon", "95", "14", "16"},  
{"017", "Croissant", "55", "15", "16"},  
{"018", "Brownie", "45", "15", "16"},  
{"019", "Chocolate Caliente", "58", "18", "16"},  
{"020", "Cafe Americano", "45", "20", "16"}
```

```
};
```

```
return producto;
```

```
}
```

```
public static String MostrarProducto(String[] vproducto) {
```

```
    String codigo = RellenarEspacios(vproducto[0], 5);
```

```
    String nombre = RellenarEspacios(vproducto[1], 30);
```

```
    String precio = RellenarEspacios(vproducto[2], 10);
```

```
    String cantidad = RellenarEspacios(vproducto[3], 10);
```

```
    String iva = RellenarEspacios(vproducto[4], 5);
```

```
    String cadena = codigo + nombre + precio + cantidad + iva;
```

```
    return cadena;
```

```
}
```

```
public static String MostrarLista(String[][] vproductos) {
```

```
    String salida = "";
```

```
    for (int ciclo = 0; ciclo < vproductos.length; ciclo++) {
```

```
String[] vproducto = {  
    vproductos[ciclo][0],  
    vproductos[ciclo][1],  
    vproductos[ciclo][2],  
    vproductos[ciclo][3],  
    vproductos[ciclo][4]  
};  
String cadena = MostrarProducto(vproducto);  
salida = salida.concat(cadena + "\n");  
}  
return salida;  
}
```

```
public static int ExisteProducto(String codigo, String[][] vproductos) {  
    int enc, pos, tam, ciclo;  
    enc = -1;  
    pos = 0;  
    tam = vproductos.length;  
    for (ciclo = 0; ciclo < tam; ciclo++) {  
        if (vproductos[ciclo][0].compareTo(codigo.trim()) == 0) {  
            enc = pos;  
        }  
        pos++;  
    }  
    return enc;  
}
```

```
public static double ObtenerIva(String codigo, String[][] productos) {
```

```
    int posicion;
```

```
    posicion = ExisteProducto(codigo, productos);
```

```
    if (posicion > -1) {
```

```
        return Double.parseDouble(productos[posicion][4]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
public static int DescontarStock(
```

```
    String[][] vproductos,
```

```
    String codigo,
```

```
    int cantidad) {
```

```
    int posicion;
```

```
    posicion = ExisteProducto(codigo, vproductos);
```

```
    if (posicion == -1) {
```

```
        return -2;
```

```
    }
```

```
    int stock = Integer.parseInt(vproductos[posicion][3]);
```

```
    if (stock == 0) {
```

```
        return 0;
    }
    if (cantidad > stock) {
        return -1;
    }
    stock = stock - cantidad;
    vproductos[posicion][3] = String.valueOf(stock);
    return 1;
}

public static void ModificarProducto(String[][] vproductos) throws IOException {
    String codigo, precio;
    int posicion;
    String info = MostrarLista(vproductos);
    codigo = Leer(info + "\nIntroduce el codigo del producto a modificar");
    if (codigo != null) {
        posicion = ExisteProducto(codigo, vproductos);
        if (posicion > -1) {
            String[] vproducto = {
                vproductos[posicion][0],
                vproductos[posicion][1],
                vproductos[posicion][2],
                vproductos[posicion][3],
                vproductos[posicion][4]
            };
            precio = LeerValidado(
```

```
"\nIntroduce el precio de " + MostrarProducto(vproducto) + " ",
2
);
vproductos[posicion][2] = precio;
System.out.println("precio actualizado");
} else {
    System.out.println("no existe el codigo");
}

} else {
    System.out.println("dato nulo");
}
}

public static void MenuProductos (String[][] vproductos) throws IOException
{
    String[] datosmenuproductos = { "1.-Modificar ", "2.-Listado ", "3.-Salida " };
    String opcion = "0";
    do
    {
        opcion = DesplegarMenu("Opciones de Productos",
        datosmenuproductos);
        if (opcion == null)
            System.out.println("opcion incorrecta ");
        else
            switch (opcion)
```

```
{  
    case "1": ModificarProducto(vproductos); break;  
    case "2": System.out.println(MostrarLista(vproductos)); break;  
  
        case "3":  
            System.out.println("Salida del Sistema "); break;  
    default: System.out.println("No existe esta opcion "); break;  
}  
}  
  
    while (opcion.compareTo("3") != 0);  
}
```

```
/**  
 * Crea e inicializa el arreglo para las ventas.  
 *  
 * @return Una matriz bidimensional que representa las ventas.  
 */  
public static String[][] CrearVenta() {  
    return new String[tamventas][5];  
}  
  
/**  
 * Obtiene el ID del último ticket registrado en el arreglo de ventas.  
 * @return El ID del último ticket, o "000" si no hay tickets registrados.  
 */
```

```
public static String UltimoTicket(int pos, String[][] mventa) {  
    String idticket = "000"; // Inicializa el ID del ticket como "000" por defecto  
    if (pos > -1) { // Verifica si la posición es válida  
        idticket = mventa[pos][0]; // Asigna el ID del ticket desde el arreglo de ventas  
    }  
    return idticket; // Devuelve el ID del último ticket  
}
```

```
/**  
 * Crea e inicializa el arreglo para los tickets.  
 *  
 * @return Una matriz bidimensional que representa los tickets.  
 */  
public static String[][] CrearTicket() {  
    return new String[20][4];  
}
```

```
/**  
 * Verifica si un código de producto existe en el ticket.  
 *  
 * @param mticket La matriz que representa el ticket.  
 * @param codigo El código del producto a buscar.  
 * @return La posición del producto en el ticket si existe, o -1 si no se  
 * encuentra.
```

*/

```
public static int ExisteTicketCodigo(String[][] mticket, String codigo) {  
    int enc = -1; // Inicializa la variable de posición como -1 (no encontrado)  
    int pos = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket  
    System.out.println(" buscando " + codigo + " tamaño arreglo " + pos);  
    for (int ciclo = 0; ciclo <= pos; ciclo++) {  
        // Compara el código del producto con el código proporcionado  
        if (mticket[ciclo][0].compareTo(codigo.trim()) == 0) {  
            enc = ciclo; // Asigna la posición donde se encontró el producto  
            return enc; // Retorna la posición encontrada  
        }  
    }  
    return enc; // Retorna -1 si no se encuentra el producto  
}
```

/**

```
* Inserta un producto en el ticket de ventas. Si el producto ya existe en el  
* ticket, incrementa su cantidad. Si no existe, lo agrega como un nuevo  
* producto.  
*  
* @param mticket La matriz que representa el ticket de ventas.  
* @param datos Un arreglo que contiene los datos del producto a insertar.  
* @param tamticket El tamaño máximo permitido para el ticket.  
* @return `true` si el producto se insertó o actualizó correctamente, `false`
```

* si el ticket está lleno.

*/

```
public static boolean InsertarProductoTicket(String[][] mticket, String[] datos, int tamticket) {  
    boolean sucedio = true; // Indica si la operación fue exitosa  
  
    int posticket = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket  
  
    int enc = ExisteTicketCodigo(mticket, datos[0]); // Verifica si el producto ya existe en el ticket  
  
    if (posticket < tamticket) { // Verifica si hay espacio en el ticket  
        if (enc > -1) { // Si el producto ya existe en el ticket  
            int cantidadactual = Integer.parseInt(mticket[enc][3]); // Obtiene la cantidad actual del  
producto  
            mticket[enc][3] = String.valueOf(cantidadactual + 1); // Incrementa la cantidad  
        } else { // Si el producto no existe en el ticket  
            posticket++; // Incrementa la posición del ticket  
            mticket[posticket][0] = datos[0]; // Asigna el código del producto  
            mticket[posticket][1] = datos[1]; // Asigna el nombre del producto  
            mticket[posticket][2] = datos[2]; // Asigna el precio del producto  
            mticket[posticket][3] = datos[3]; // Asigna la cantidad del producto  
            System.out.println("Insertando: No existe el producto en el ticket");  
        }  
    } else {  
        sucedio = false; // Indica que no se pudo insertar el producto porque el ticket está lleno  
    }  
  
    return sucedio; // Devuelve el resultado de la operación  
}
```

/**

* Calcula el total por producto multiplicando el precio por la cantidad.

*

* @param precio El precio unitario del producto como cadena.

* @param cantidad La cantidad del producto como cadena.

* @return El total calculado como una cadena con dos decimales.

*/

```
public static String TotalProducto(String precio, String cantidad) {
```

```
    double total = Double.parseDouble(precio) * Double.parseDouble(cantidad); // Calcula el total
```

```
    return String.format("%.2f", total); // Formatea el total con dos decimales
```

```
}
```

/**

* Muestra los detalles de un producto en el ticket de ventas. Incluye el

* código, nombre, precio, cantidad y el total por producto.

*

* @param mticket La matriz que representa el ticket de ventas.

* @param pos La posición del producto en el ticket.

* @return Una cadena con los detalles del producto en el ticket.

*/

```
public static String MostrarProductoTicket(String[][] mticket, int pos) {
```

```
    String codigo = RellenarEspacios(mticket[pos][0], 5); // Código del producto
```

```
    String producto = RellenarEspacios(mticket[pos][1], 30); // Nombre del producto
```

```

String precio = RellenarEspacios(mticket[pos][2], 10); // Precio del producto

String cantidad = RellenarEspacios(mticket[pos][3], 5); // Cantidad del producto

String totalproducto = RellenarEspacios(TotalProducto(mticket[pos][2], mticket[pos][3]), 10); //

Total por

// producto

String cadena = codigo.concat(producto + precio + cantidad + totalproducto); // Concatenación

de los

// detalles

return cadena; // Devuelve la cadena con los detalles del producto

}

/**
 * Muestra el contenido del ticket de ventas. Recorre el ticket y genera una
 * cadena con los detalles de cada producto.
 *
 * @param mticket La matriz que representa el ticket de ventas.
 * @return Una cadena con el detalle de los productos en el ticket.
 */

public static String MostrarTicket(String[][] mticket) {

    String salida = "";

    int pos = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el

ticket

    for (int ciclo = 0; ciclo <= pos; ciclo++)

        salida = salida.concat(MostrarProductoTicket(mticket, ciclo) + "\n"); // Agrega el

detalle de cada

// producto

return salida; // Devuelve la cadena con el contenido del ticket

```

}

/**

* Calcula el subtotal del ticket sumando el total de cada producto en el

* ticket.

*

* @param mticket La matriz que representa el ticket de ventas.

* @return El subtotal del ticket como un valor de tipo double.

*/

```
public static double SubTotalTicket(String[][] mticket) {
```

```
    double subtotal = 0;
```

```
    int pos = ObtenerUltimaPosicion(mticket); // Obtiene la última posición válida en el ticket
```

```
    for (int ciclo = 0; ciclo <= pos; ciclo++)
```

```
        subtotal = subtotal + Double.parseDouble(TotalProducto(mticket[ciclo][2], mticket[ciclo][3])); // Suma el  
    total
```

```
        // de cada
```

```
        // producto
```

```
    return subtotal; // Devuelve el subtotal calculado
```

```
}
```

/**

* Calcula el IVA (Impuesto al Valor Agregado) del ticket. Si el subtotal del

* ticket es mayor a 0, calcula el 16% del subtotal. Si el subtotal es 0 o

* menor, devuelve -1 indicando un error.

*

* @param mticket La matriz que representa el ticket de ventas.

* @return El monto del IVA calculado o -1 si el subtotal es inválido.

*/

```
public static double IvaTicket(String[][] mticket, String[][] productos) {
```

```
    double ivatotal = 0;
```

```
    int pos = ObtenerUltimaPosicion(mticket);
```

```
    for (int ciclo = 0; ciclo <= pos; ciclo++) {
```

```
        String codigo = mticket[ciclo][0];
```

```
        double iva = ObtenerIva(codigo, productos);
```

```
        if (iva != 0) {
```

```
            double precio = Double.parseDouble(mticket[ciclo][2]);
```

```
            double cantidad = Double.parseDouble(mticket[ciclo][3]);
```

```
            double totalproducto = precio * cantidad;
```

```
            ivatotal = ivatotal + ((iva / 100) * totalproducto);
```

```
        }
```

```
    }
```

```
    return ivatotal;
```

```
}
```

```
/**
```

```
 * Calcula el total del ticket, incluyendo el subtotal y el IVA.
```

```
 *
```

```
 * @param mticket La matriz que representa el ticket de ventas.
```

```
 * @return El total del ticket, que incluye el subtotal y el IVA.
```

```
*/
```

```
public static double TotalTicket(String[][] mticket, String[][] productos) {
```

```
    double total = SubTotalTicket(mticket);
```

```
if (total > 0) {  
    total = IvaTicket(mticket, productos) + total;  
}  
return total;  
}  
  
/**  
 * Muestra el ticket de venta con el detalle de los productos, subtotal, IVA y  
 * total. Incluye la fecha y el número de ticket en el encabezado.  
 *  
 * @param mticket La matriz que representa el ticket de ventas.  
 * @param idticket El ID del ticket actual.  
 * @param fecha La fecha de la venta.  
 * @return Una cadena con el detalle del ticket de venta.  
 */  
public static String MostrarTicketVenta(  
    String[][] mticket,  
    String idticket,  
    String fecha,  
    String[][] productos) {  
    String salida = "";  
    String subtotal = String.format("%.2f",  
        SubTotalTicket(mticket));  
    String iva = String.format("%.2f",  
        IvaTicket(mticket, productos));  
    String total = String.format("%.2f",
```

```
TotalTicket(mticket, productos));

salida = "Fecha " + fecha + "Ticket No." + idticket;
salida = salida + "\n" + MostrarTicket(mticket);
salida = salida + "\n \n El total sin iva " + subtotal;
salida = salida + "\n el iva total es " + iva;
salida = salida + "\n el total de la venta fue " + total;
return salida;
}
```

```
/**
```

```
* Muestra la lista de productos disponibles para la venta. Recorre el
* inventario de productos y verifica si hay existencia disponible (cantidad
* mayor a 0). Si el producto está disponible, lo agrega a la lista de salida.
*
* @param vproductos La matriz que representa el inventario de productos.
* @return Una cadena con la lista de productos disponibles para la venta.
*/
```

```
public static String MostrarListaProductosVenta(String[][] vproductos) {
    String salida = "";
    for (int ciclo = 0; ciclo < vproductos.length; ciclo++) {
        int existencia = Integer.parseInt(vproductos[ciclo][3]); // Obtiene la cantidad disponible del producto
        if (existencia > 0) {
            String[] vproducto = vproductos[ciclo].clone(); // Clona los datos del producto
            String cadena = MostrarProducto(vproducto); // Genera la cadena con los datos del producto
        }
    }
    return salida;
}
```

```
        salida = salida.concat(cadena + "\n"); // Agrega el producto a la lista de salida
    }
}

return salida; // Devuelve la lista de productos disponibles
}

/**
 * Captura un producto para agregar al ticket de ventas. Muestra la lista de
 * productos disponibles para la venta, solicita el código del producto,
 * verifica su existencia y disponibilidad en el inventario, y lo agrega al
 * ticket si es posible. Si el producto no está disponible o el código no
 * existe, muestra mensajes de error correspondientes.
 *
 * @param mticket La matriz que representa el ticket de ventas.
 * @param mproductos La matriz que representa el inventario de productos.
 * @param idticket El ID del ticket actual.
 * @param tamticket El tamaño máximo permitido para el ticket.
 * @throws IOException Si ocurre un error en la entrada/salida.
 */
public static void CapturaVentaProducto(String[][] mticket, String[][] mproductos, String idticket, int tamticket)
    throws IOException {
    String codigo, info;
    info = MostrarListaProductosVenta(mproductos);
    codigo = Leer(info + "\nIntroduce el codigo del producto");
    if (codigo != null) {
```

```
int posp = ExisteProducto(codigo.trim(), mproductos);

if (posp > -1) {
    String[] producto = mproductos[posp].clone();
    int resultado = DescontarStock(mproductos, codigo, 1);
    if (resultado == 1) {
        System.out.println(MostrarProducto(producto));

        String[] venta = new String[4];
        venta[0] = mproductos[posp][0];
        venta[1] = mproductos[posp][1];
        venta[2] = mproductos[posp][2];
        venta[3] = "1";
        if (!InsertarProductoTicket(mticket, venta, tamticket)) {
            System.out.println("el Arreglo esta lleno");
        }
    } else if (resultado == 0) {
        System.out.println("no hay productos para venta");
    } else if (resultado == -1) {
        System.out.println("la cantidad es mayor al stock");
    } else if (resultado == -2) {
        System.out.println("el codigo no existe no se puede agregar");
    }
} else {
    System.out.println("el codigo no existe no se puede agregar");
}

} else {
```

```
System.out.println("dato nulo");

}

}

/**
 * Elimina un producto del ticket. Si el producto no es el último en la lista,
 * desplaza los elementos posteriores para llenar el espacio vacío. Si el
 * producto es el último, simplemente lo elimina.
 *
 * @param mticket La matriz que representa el ticket de ventas.
 * @param pos La posición del producto en el ticket que se desea eliminar.
 */
public static void RemoverProductoTicket(String[][] mticket, int pos) {
    int tam = ObtenerUltimaPosicion(mticket);

    if (tam > pos) {
        for (int i = pos; i < tam; i++) {
            mticket[i][0] = mticket[i + 1][0];
            mticket[i][1] = mticket[i + 1][1];
            mticket[i][2] = mticket[i + 1][2];
            mticket[i][3] = mticket[i + 1][3];
        }
    }

    mticket[tam][0] = null;
```

```
mticket[tam][1] = null;
mticket[tam][2] = null;
mticket[tam][3] = null;
}

/**
 * Elimina un producto del ticket. Si la cantidad del producto es mayor a 1,
 * simplemente decrementa la cantidad. Si la cantidad es 1, elimina el producto
 * completamente del ticket.
 *
 * @param mticket La matriz que representa el ticket de ventas.
 * @param pos La posición del producto en el ticket que se desea eliminar.
 */
public static void EliminarProductoTicket(String[][] mticket, int pos) {
    int cantidad = Integer.parseInt(mticket[pos][3]); // Obtiene la cantidad del producto en el ticket
    if (cantidad > 1) {
        mticket[pos][3] = String.valueOf(cantidad - 1); // Decrementa la cantidad si es mayor a 1
    } else {
        RemoverProductoTicket(mticket, pos); // Elimina el producto completamente si la cantidad es 1
    }
}

/**
 * Elimina un producto del ticket y actualiza el inventario. Muestra los
 * productos en el ticket, solicita el código del producto a eliminar, y si el
```

* producto existe en el ticket, lo elimina y actualiza la cantidad en el

* inventario.

*

* @param mticket La matriz que representa el ticket de ventas.

* @param mproductos La matriz que representa el inventario de productos.

* @throws IOException Si ocurre un error en la entrada/salida.

*/

```
public static void Eliminar(String[][] mticket, String[][] mproductos) throws IOException {
```

```
    String codigo, info;
```

```
    info = MostrarTicket(mticket);
```

```
    codigo = Leer(info + "\nIntroduce el codigo del producto");
```

```
    if (codigo != null) {
```

```
        int pos = ExisteTicketCodigo(mticket, codigo);
```

```
        if (pos > -1) {
```

```
            int posproducto = ExisteProducto(codigo, mproductos);
```

```
            String nuevacantidad = String.valueOf((Integer.valueOf(mproductos[posproducto][3]) + 1));
```

```
            mproductos[posproducto][3] = nuevacantidad;
```

```
            mproductos[posproducto][3] = nuevacantidad;
```

```
            EliminarProductoTicket(mticket, pos);
```

```
        } else {
```

```
            System.out.println("no existe el producto en el ticket");
```

```
        }
```

```
    } else {
```

```
        System.out.println("dato nulo");
```

```
    }
```

```
}
```

/**

- * Agrega los productos de un ticket a la matriz de ventas. Recorre el ticket y
- * transfiere cada producto a la matriz de ventas, asignando el ID del ticket a
- * cada registro.

*

- * @param mticket La matriz que representa el ticket de venta.
- * @param mventa La matriz que representa las ventas realizadas.
- * @param idticket El ID del ticket que se está procesando.

*/

```
public static void AgregarProductoAVenta(String[][] mticket, String[][] mventa, String idticket) {  
    int posventas = ObtenerUltimaPosicion(mventa); // Obtiene la última posición ocupada en ventas  
    int posticket = ObtenerUltimaPosicion(mticket); // Obtiene la última posición ocupada en el ticket  
    for (int i = 0; i <= posticket; i++) {  
        if (mticket[i][0] != null) { // Verifica que la fila del ticket no esté vacía  
            posventas++; // Incrementa la posición en ventas  
            mventa[posventas][0] = idticket; // Establece el ID del ticket  
            mventa[posventas][1] = mticket[i][0]; // Código del producto  
            mventa[posventas][2] = mticket[i][1]; // Nombre del producto  
            mventa[posventas][3] = mticket[i][2]; // Precio del producto  
            mventa[posventas][4] = mticket[i][3]; // Cantidad del producto  
        }  
    }  
}
```

/**

* Realiza el proceso de pago de un ticket, agregando los productos del ticket a

* las ventas. Verifica si hay suficiente espacio en el arreglo de ventas antes

* de realizar la operación.

*

* @param idticket El ID del ticket que se está pagando.

* @param mventa La matriz que representa las ventas.

* @param mticket La matriz que representa el ticket actual.

*/

```
public static void Pagar(String idticket, String[][] mventa, String[][] mticket) {
```

```
    int posventas = ObtenerUltimaPosicion(mventa); // Obtiene la última posición ocupada en las ventas
```

```
    int post = ObtenerUltimaPosicion(mticket); // Obtiene la última posición ocupada en el ticket
```

```
    // Verifica si hay espacio suficiente en el arreglo de ventas
```

```
    if ((posventas + post) < 100) {
```

```
        AgregarProductoAVenta(mticket, mventa, idticket); // Agrega los productos del ticket a las  
ventas
```

```
    } else {
```

```
        System.out.println("Desbordamiento de Memoria de ventas"); // Mensaje de error si no hay  
espacio
```

```
    }
```

```
}
```

/**

* Realiza la devolución de los productos de un ticket al inventario. Recorre el

* ticket y, por cada producto, incrementa la cantidad en el inventario.

*

* @param mticket La matriz que representa el ticket de venta.

* @param mproductos La matriz que representa el inventario de productos.

*/

```
public static void DevolucionTicket(String[][] mticket, String[][] mproductos) {
```

```
    int posmticket = ObtenerUltimaPosicion(mticket); // Obtiene la última posición ocupada en el ticket
```

```
    for (int pos = 0; pos <= posmticket; pos++) {
```

```
        String codigo = mticket[pos][0]; // Obtiene el código del producto del ticket
```

```
        int posp = ExisteProducto(codigo.trim(), mproductos); // Busca el producto en el inventario
```

```
        if (posp > -1) { // Si el producto existe en el inventario
```

```
            int cant = Integer.parseInt(mticket[pos][3]) + Integer.parseInt(mproductos[posp][3]); //
```

Suma las

```
                                // cantidades
```

```
            mproductos[posp][3] = String.valueOf(cant); // Actualiza la cantidad en el inventario
```

```
        }
```

```
    }
```

```
}
```

```
public static void CancelarVenta(String[][] mticket, String[][] mproductos) {
```

```
    DevolucionTicket(mticket, mproductos);
```

```
    for (int i = 0; i < mticket.length; i++) {
```

```
        for (int j = 0; j < mticket[i].length; j++) {
```

```
            mticket[i][j] = null;
```

```
        }
```

```
    }
```

```
}
```

/**

* Muestra el menú del punto de venta, permitiendo agregar, eliminar, listar

* productos, realizar el pago del ticket o salir del sistema. Gestiona las

* operaciones relacionadas con el ticket y las ventas.

*

* @param ventas La matriz que representa las ventas realizadas.

* @param idticket El ID del ticket actual.

* @param productos La matriz que representa el inventario de productos.

* @throws IOException Si ocurre un error en la entrada/salida.

*/

```
public static void MenuPuntoVenta(String[][] ventas, String idticket, String[][] productos) throws IOException {
```

```
    String opcion, membrete;
```

```
    boolean pago = false;
```

```
    int tamticket = 50;
```

```
    String[][] Vticket = new String[tamticket][4];
```

```
    idticket = IdTicketSiguiente(idticket);
```

```
    String fechadia = Fecha();
```

```
    opcion = "";
```

```
    do {
```

```
        membrete = "Fecha del Dia " + fechadia + " Ticket No " + idticket;
```

```
        membrete = membrete + "\n-----\n";
```

```
String Tickettexto = MostrarTicket(Vticket).trim();
if (!Tickettexto.trim().isEmpty()) {
    membrete = membrete + "\n" + Tickettexto + "\n";
}

String[] datosmenu = { "1.-Agregar ", "2.-Eliminar ", "3.-Listado ", "4.-Pagar ", "5.-Salida " };
opcion = DesplegarMenu(membrete + "\n Menu de Punto de Venta", datosmenu);

if (opcion == null) {
    System.out.println("dato incorrecto introducido");
} else {
    switch (opcion) {
        case "1":
            CapturaVentaProducto(Vticket, productos, idticket, tamticket);
            break;
        case "2":
            Eliminar(Vticket, productos);
            break;
        case "3":
            if (ObtenerUltimaPosicion(Vticket) > -1) {
                System.out.println(MostrarTicket(Vticket));
            }
            break;
        case "4":
            System.out.println(MostrarTicketVenta(Vticket, idticket, fechadia, productos).trim());
```

```
Pagar(idticket, ventas, Vticket);

pago = true;

opcion = "5";

break;

case "5":

    System.out.println("Salida del Ventas");

    if (!pago) {

        System.out.println("No pago el ticket");

        CancelarVenta(Vticket, productos);

        System.out.println("eliminando tickte " + idticket);

    }

    break;

default:

    System.out.println("No existe esta opcion");

    break;

}

}

} while (opcion.compareTo("5") != 0);

}

/**
 * Muestra los detalles de una venta específica. Incluye el ID del ticket, el
 * código del producto, el nombre del producto, el precio y la cantidad.
 *
 * @param venta Un arreglo que contiene los datos de la venta: [0] - ID del
 * ticket [1] - Código del producto [2] - Nombre del producto [3] -
```

```

*      Precio del producto [4] - Cantidad del producto

* @return Una cadena con los detalles de la venta formateados.

*/

public static String MostrarVenta(String[] venta) {

    String idticket = RellenarEspacios(venta[0], 6); // ID del ticket

    String codigo = RellenarEspacios(venta[1], 5); // Código del producto

    String producto = RellenarEspacios(venta[2], 30); // Nombre del producto

    String precio = RellenarEspacios(venta[3], 10); // Precio del producto

    String cantidad = RellenarEspacios(venta[4], 10); // Cantidad del producto

    String cadena = idticket.concat(codigo + producto + precio + cantidad); // Concatenación de los detalles

    return cadena; // Devuelve la cadena con los detalles de la venta

}

/**
 * Muestra la lista de todas las ventas realizadas. Recorre la matriz de ventas
 * y genera una cadena con los detalles de cada venta.
 *
 * @param ventas La matriz que representa las ventas realizadas.
 * @return Una cadena con el detalle de todas las ventas.
 */

public static String MostrarListaVentas(String[][] ventas) {

    int posventas = ObtenerUltimaPosicion(ventas); // Obtiene la última posición válida en la matriz
de ventas

    String salida = "";

    for (int ciclo = 0; ciclo <= posventas; ciclo++) {

        // Crea un arreglo con los datos de una venta específica

```

```
String[] venta = { ventas[ciclo][0], ventas[ciclo][1], ventas[ciclo][2], ventas[ciclo][3],  
                  ventas[ciclo][4] };  
  
String cadena = MostrarVenta(venta); // Genera una cadena con los detalles de la venta  
salida = salida.concat(cadena + "\n"); // Agrega la cadena al resultado final  
  
}  
  
return salida; // Devuelve la cadena con el detalle de todas las ventas  
  
}
```

```
public static void AgregarStock(String[][] vproductos) throws IOException {  
  
    String codigo, cantidad;  
  
    int posicion;  
  
  
    String info = MostrarLista(vproductos);  
    codigo = LeerValidado(info + "\nIntroduce el codigo del producto a modificar", 1);  
  
    if (codigo != null) {  
  
        posicion = ExisteProducto(codigo, vproductos);  
  
        if (posicion > -1) {  
  
            String[] vproducto = {  
                vproductos[posicion][0],  
                vproductos[posicion][1],  
                vproductos[posicion][2],  
                vproductos[posicion][3],  

```

```
vproductos[posicion][4]
    };

    cantidad = LeerValidado("\nIntroduce la cantidad de stock a agregar ", 1);

    int stockActual = Integer.parseInt(vproductos[posicion][3]);
    int stockNuevo = Integer.parseInt(cantidad);

    vproductos[posicion][3] = String.valueOf(stockActual + stockNuevo);

    System.out.println("stock actualizado");

    } else {
        System.out.println("no existe el codigo");
    }

    } else {
        System.out.println("dato nulo");
    }
}

public static void MenuInventario (String[][] vproductos ) throws IOException
{
    String[] datosmenuinventario = { "1.-Listado ", "2.-Agregar ", "3.-Salida " };
    String opcion = "0";
```

```
do
{
opcion = DesplegarMenu("Opciones de Inventarios",datosmenuinventario);
if (opcion == null)
    System.out.println("opcion incorrecta ");
else
switch (opcion)
{
    case "1": System.out.println(MostrarLista(vproductos)); break;
    case "2": AgregarStock(vproductos); break;
    case "3": System.out.println("Salida del Sistema "); break;
default: System.out.println("No existe esta opcion "); break;
}
}

while (opcion.compareTo("3") != 0);
}

public static void MenuPrincipal(String[][] vproductos,String[][] vventas) throws IOException
{ String[] datosmenuprincipal = { "1.-Productos ", "2.-Punto de Venta ", "3.- Inventario", "4.-Ventas", "5.-Salida "
};

String opcion = "0";

String idticket;

do
{
idticket=      ObtenerUltimoValorVentas(vventas);

opcion = DesplegarMenu("\nMenu de Punto de Venta Cafeteria Coffee House\n",
datosmenuprincipal);
```

```
if (opcion == null)

    System.out.println("opcion incorrecta ");

else

switch (opcion)

{

//falta adecuar el idticket que revise ventas y si esta vacio sea 000 si no el ultimo del arreglo

    case "1": MenuProductos(vproductos); break;

    case "2": MenuPuntoVenta(vventas,idticket,vproductos); break;

    case "3": MenuInventario(vproductos); break;

    case "4": System.out.println(MostrarListaVentas(vventas)); break;

    case "5":

System.out.println("Salida del Sistema "); break;

default: System.out.println("No existe esta opcion "); break;

}

}

    while (opcion.compareTo("5") != 0);

}

/**

* Obtiene el último valor del ID de las ventas registradas. Recorre la matriz

* de ventas para encontrar la última posición válida y devuelve el valor del ID

* de esa posición. Si no hay ventas registradas, devuelve "000" como valor

* predeterminado.

*
```

* @param ventas La matriz que representa las ventas realizadas.

* @return El último valor del ID de las ventas o "000" si no hay registros.

*/

```
public static String ObtenerUltimoValorVentas(String[][] ventas) {  
    int ultimaposicion = ObtenerUltimaPosicion(ventas); // Obtiene la última posición válida en la matriz de  
    ventas  
  
    String ultimoValor = "000"; // Valor predeterminado si no hay registros  
  
    if (ultimaposicion >= 0) {  
        ultimoValor = ventas[ultimaposicion][0]; // Asigna el ID de la última venta registrada  
    }  
  
    return ultimoValor; // Devuelve el último valor del ID de las ventas  
}  
  
public static void main(String[] args) throws IOException  
{  
    productos = CargarProductos();  
    ventas=CrearVenta();  
    MenuPrincipal(productos,ventas);  
}
```